สรุปเนื้อหาเตรียมสอบ Final

Parallel Algorithms with CUDA

อ้างอิง: `pram.pdf` (Chatterjee & Prins), `parallel.pdf` (Blelloch et al.)
และแบบฝึกหัด P4–P7 ในโฟลเดอร์ `homework`

ขอบเขตการวัดผล (ตามที่ประกาศ)

1. **กรอบวัดเกี่ยวกับ parallel model**
   - PRAM model — optimal speedup
   - Work-Step (WS) model — ออกแบบ parallel algorithm แล้วเขียนเป็น WS pseudocode, work-optimal condition และ minimum step
2. **Parallel algorithms**: Reduce, Scan, Histogram (basic building blocks) — อาจให้ปัญหาใหม่แล้วให้ *decompose* เป็น building block เหล่านี้
3. **จาก high-level idea สู่ low-level implementation (CUDA)** — รวมถึงการเสนอ idea ในการ fine-tune implementation โดยอิงหลักการที่ *วัดได้* (measurable): maximize throughput และ utilize bandwidth

Contents

1 PRAM Model **4**

1.1 นิยามของ PRAM 4

1.2 ประเภทของ PRAM ตามการเข้าถึง memory พร้อมกัน 4

1.3 ตัววัดประสิทธิภาพบน PRAM 4

1.4 ตัวอย่าง: Sum (หรือ product, max) ของ n ค่าบน PRAM 5

1.5 ข้อจำกัดของ PRAM ในทางปฏิบัติ 5

2 Work-Step (Work-Time) Model **5**

2.1 แนวคิด 6

2.2 ตัววัดความซับซ้อน 6

2.3 Brent's Theorem: จาก WS สู่ PRAM 6

2.4 Work-optimal condition 6

2.5 Minimum step และเงื่อนไข optimal speedup . . . 7
2.6 ลักษณะของ parallel solution ที่ดีบน WS model . . . 7
2.7 การเขียน WS pseudocode . . . 8
2.8 เทคนิคการออกแบบที่สำคัญ . . . 8

3 Basic Building Blocks: Reduce, Scan, Histogram 9

3.1 Reduce . . . 9
3.2 Scan (Prefix Sum) . . . 9
3.2.1 Hillis–Steele inclusive scan (1986) . . . 10
3.2.2 Blelloch exclusive scan (1990) — work-efficient . . . 10
3.2.3 Recursive scan (Algorithm 4 ใน `pram.pdf`) . . . 11
3.2.4 Primitives ที่สร้างจาก Scan . . . 11
3.3 Histogram . . . 11
3.4 การ Decompose ปัญหาใหม่เป็น Building Blocks . . . 12

4 Algorithms ที่สร้างจาก Building Blocks: Filter, Flatten, Search, Merge, K-th Smallest 13

4.1 Filter (Pack / Stream compaction) . . . 13
4.2 Flatten . . . 14
4.3 Search (Sorted search) . . . 15
4.4 K-th of two sorted arrays (`kth`, co-rank) . . . 16
4.5 Merge . . . 16
4.6 K-th smallest (Selection) . . . 17
4.7 สรุป . . . 19

5 จาก High-level Idea สู่ Low-level CUDA Implementation 19

5.1 CUDA Programming Model โดยย่อ . . . 19
5.2 การ Map แนวคิด WS/PRAM ลง CUDA . . . 20
5.3 Reduce บน CUDA . . . 20
5.3.1 Practice 4.1 — ใช้ global memory อย่างเดียว . . . 20
5.3.2 Practice 4.2 — per-block shared memory . . . 20
5.3.3 ขั้นตอนการ fine-tune reduce (ตาม principle) . . . 21
5.4 Scan บน CUDA . . . 22

5.4.1 Hillis–Steele (single block, double buffer) . . . 22
5.4.2 Blelloch (single block, $n = 2\times$`blockDim`) . . . 22
5.4.3 Scan ขนาดใหญ่ (หลาย block) — hierarchical scan . . . 23
5.5 Histogram บน CUDA . . . 23
5.5.1 Naive: global atomics . . . 23
5.5.2 Practice 7.2 — Privatized histogram ใน shared memory . . . 23
5.5.3 Practice 7.1 — 4096 bins, saturate ที่ 127 . . . 24
5.5.4 เทคนิค fine-tune histogram เพิ่มเติม . . . 24

6 หลักการ Fine-tune: วัดได้, Maximize Throughput, Utilize Bandwidth 24
6.1 หลักการต้องวัดได้ (measurable) . . . 24
6.2 Maximize throughput . . . 25
6.3 Utilize bandwidth . . . 25
6.4 Occupancy: วิธีคำนวณ . . . 26
6.5 Checklist สำหรับตอบข้อสอบ ”เสนอ idea fine-tune” . . . 26

7 สรุปสูตรและตารางอ้างอิงเร็ว 26

1 PRAM Model

1.1 นิยามของ PRAM

PRAM (Parallel Random Access Machine) เป็นส่วนขยายของ RAM model ของการคำนวณแบบ sequential (synchronous PRAM ตาม `pram.pdf`):

1. มี processor p ตัว เชื่อมต่อกับ **shared memory** ร่วมกันหนึ่งชุด
2. processor แต่ละตัวมีหมายเลขประจำตัว (processor id) $1 \le i \le p$ ไม่ซ้ำกัน
3. ทุก processor รันโปรแกรมเดียวกันแบบ **SIMD** (lock-step): แต่ละคำสั่งใช้เวลา 1 หน่วยเวลา ไม่ว่าจะมี processor กี่ตัว
4. แต่ละ processor มี flag ส่วนตัวสำหรับระบุว่า active หรือไม่ในคำสั่งนั้น (ใช้ทำ conditional execution)

PRAM **ไม่สนใจ** ต้นทุนของ synchronization และ communication — จึงเป็น baseline ของ concurrency: *ถ้าหา parallel algorithm ที่ดีบน PRAM ไม่ได้ ก็จะหาไม่ได้ในโลกจริงเช่นกัน*

1.2 ประเภทของ PRAM ตามการเข้าถึง memory พร้อมกัน

Model	ความหมาย
EREW	Exclusive Read Exclusive Write — ห้ามอ่านหรือเขียนตำแหน่งเดียวกันพร้อมกัน
CREW	Concurrent Read Exclusive Write — อ่านพร้อมกันได้ เขียนพร้อมกันไม่ได้
CRCW	Concurrent Read Concurrent Write — แบ่งย่อยตามวิธีจัดการการเขียนชนกัน:
	Common: เขียนพร้อมกันได้เมื่อทุกตัวเขียนค่าเดียวกัน
	Arbitrary: เลือกค่าใดค่าหนึ่งแบบสุ่ม
	Priority: processor ที่ id น้อยสุดชนะ
	Combining: เก็บผลรวม (เช่น +, max) ของค่าทั้งหมดที่เขียน

ความแรงของ model: EREW $\le$ CREW $\le$ Common $\le$ Arbitrary $\le$ Priority $\le$ Combining. EREW สามารถ simulate Priority CRCW ได้โดยช้าลง $O(\log p)$ เท่า (ใช้การ sort). การ broadcast ค่าเดียวให้ p processor บน EREW ใช้เวลา $\Theta(\log p)$ (copy แบบ doubling).

1.3 ตัววัดประสิทธิภาพบน PRAM

ให้ $T^*(n)$ คือเวลาของ **optimal (หรือ best known) sequential algorithm** และ $T_p(n) = T(n, p)$ คือเวลาของ parallel algorithm บน p processors

นิยามสำคัญ

$$\text{Speedup} \quad S_p(n) = \frac{T^*(n)}{T_p(n)}$$

$$\text{Cost (processor–time product)} \quad C_p(n) = p \cdot T_p(n)$$

$$\text{Efficiency} \quad E_p(n) = \frac{S_p(n)}{p} = \frac{T^*(n)}{pT_p(n)}$$

Optimal speedup: $S_p(n) = \Theta(p)$ (หรือ $E_p = \Theta(1)$)
Cost-optimal: $C_p(n) = \Theta(T^*(n))$ — ทั้งสองเงื่อนไขนี้สมมูลกัน

ทำไม speedup เกิน $O(p)$ ไม่ได้? ถ้ามี parallel algorithm ที่ได้ speedup $\omega(p)$ เราจะสามารถ *simulate* มันบน processor ตัวเดียว (ให้ processor ตัวเดียวทำงานแทน p ตัวทีละขั้น) ได้เวลา $p \cdot T_p(n) = o(T^*(n))$ ซึ่งขัดแย้งกับข้อสมมุติว่า $T^*(n)$ เป็น optimal sequential time.

1.4 ตัวอย่าง: Sum (หรือ product, max) ของ n ค่าบน PRAM

Idea (ใช้ใน Quiz 2 ข้อ 3 — การคูณ n จำนวน):

1. แบ่ง input เป็น p block ขนาด n/p; processor i รวม block ของตัวเองแบบ sequential — ใช้เวลา $O(n/p)$
2. รวมผลลัพธ์ p ค่าด้วย balanced binary tree — ใช้เวลา $O(\log p)$

$$T_p(n) = O\left(\frac{n}{p} + \log p\right), \qquad C_p(n) = O(n + p\log p).$$

จะได้ cost-optimal (optimal speedup) เมื่อ $p\log p = O(n)$ คือ $\boxed{p = O(n/\log n)}$. เมื่อเลือก $p = n/\log n$ จะได้ $T_p = O(\log n)$, $C_p = O(n)$, speedup $= \Theta(n/\log n) = \Theta(p)$.

ข้อควรระวัง

ถ้าใช้ $p = n$ processors ทำ tree ตรง ๆ ได้ $T = O(\log n)$ แต่ cost $= n\log n$ ซึ่ง **ไม่ cost-optimal** — speedup ได้เพียง $n/\log n$ ไม่ใช่ $\Theta(p) = \Theta(n)$

1.5 ข้อจำกัดของ PRAM ในทางปฏิบัติ

PRAM สมมุติว่า memory access มี latency คงที่ ซึ่งไม่จริงเมื่อ p ใหญ่ขึ้น (ต้องมี memory bank จำนวนมาก, switching network มี latency, ขนาดทางกายภาพ). อย่างไรก็ตาม algorithm ที่ดีบน PRAM เป็นจุดเริ่มต้นที่ดี: ถ้าให้ processor จริงแต่ละตัว simulate PRAM processor หลายตัว จะสามารถ **ซ่อน memory latency** ได้ — นี่คือหลักการเดียวกับ GPU ที่มี thread จำนวนมากกว่า core มาก (latency hiding ด้วย massive multithreading).

2 Work-Step (Work-Time) Model

2.1 แนวคิด

การเขียน PRAM algorithm ต้องระบุ processor id และ scheduling เอง ซึ่งยุ่งยาก WS model (เรียกอีกชื่อว่า Work-Time, Work-Depth หรือ Work-Span) จึง *ซ่อน* จำนวน processor:

- แต่ละ step มี operation พร้อมกันได้ **ไม่จำกัดจำนวน**
- ใช้ construct `for all ... in parallel` (หรือ `forall`, `parfor`) แทนการทำงานพร้อมกัน — นี่คือ construct เดียวที่แตกต่างจาก sequential pseudocode
- ใน `forall` หนึ่งครั้ง ค่าทางขวาของ assignment ทั้งหมดถูก evaluate **ก่อน** จะเขียนค่าทางซ้าย (synchronous semantics)

2.2 ตัววัดความซับซ้อน

Work และ Step

- **Work** $W(n)$: จำนวน operation ทั้งหมด $= \sum_{i=1}^{S(n)} W_i(n)$ เมื่อ $W_i(n)$ คือจำนวน operation ใน step ที่ i (เท่ากับเวลาเมื่อรันบน processor ตัวเดียว)
- **Step** $S(n)$ (step complexity / span / depth): จำนวน parallel step (เท่ากับเวลาเมื่อมี processor ไม่จำกัด; คือความยาวของ critical path ใน DAG)
- **Parallelism** $= W(n)/S(n)$ — จำนวน processor สูงสุดที่ใช้ได้อย่างคุ้มค่า

กฎการรวม: งานที่ทำ *ต่อกัน* ⇒ บวก work, บวก step; งานที่ทำ *ขนานกัน* ⇒ บวก work, step เอาค่า **max**

2.3 Brent's Theorem: จาก WS สู่ PRAM

Theorem 1 (Brent 1974)**.** *WS algorithm ที่มี work $W(n)$ และ step $S(n)$ สามารถ simulate บน PRAM p processors ได้ในเวลา*

$$T_p(n) \;\le\; \left\lfloor \frac{W(n)}{p} \right\rfloor + S(n) \qquad \text{และ} \qquad T_p(n) \;\ge\; \max\!\left(\frac{W(n)}{p},\; S(n) \right).$$

แนวพิสูจน์: step ที่ i มี W_i operation; ใช้ p processors ทำทีละ p งานได้ใน $\lceil W_i/p \rceil \le \lfloor W_i/p \rfloor + 1$ เวลา รวมทุก step ได้ $\sum_i (\lfloor W_i/p \rfloor + 1) \le \lfloor W/p \rfloor + S$.

2.4 Work-optimal condition

Definition 1. *WS algorithm เป็น **work-optimal** (work-efficient) ถ้า $W(n) = \Theta(T^*(n))$*

ทำไม work สำคัญที่สุด? จาก Brent: $T_p = O(W(n)/p + S(n))$. ถ้า $W(n) = \omega(T^*(n))$ แล้ว เมื่อ p คงที่และ n โตขึ้น

$$S_p(n) = \frac{T^*(n)}{W(n)/p + S(n)} \;\le\; p \cdot \frac{T^*(n)}{W(n)} \;\xrightarrow{n\to\infty}\; 0$$

ในโลกจริง p มีจำกัด (เช่น จำนวน core บน GPU) แต่ n ใหญ่ขึ้นได้เรื่อย ๆ ดังนั้น algorithm ที่ไม่ work-optimal จะ **เสีย speedup แบบไม่มีขอบเขต**

Work $o(T^*(n))$ เป็นไปไม่ได้ — ถ้ามี WS algorithm ที่ $W(n) = o(T^*(n))$ เราสามารถรันบน processor ตัวเดียวได้ในเวลา $O(W(n))$ (Brent ด้วย $p = 1$, และ $S(n) \le W(n)$) จึงได้ sequential algorithm ที่เร็วกว่า optimal — ขัดแย้ง.

2.5 Minimum step และเงื่อนไข optimal speedup

เมื่อ algorithm เป็น work-optimal แล้ว

$$T_p(n) = O\left(\frac{T^*(n)}{p} + S(n)\right), \qquad S_p(n) = \frac{T^*(n)}{T_p(n)} = \Theta(p) \quad \text{ตราบที่} \quad \boxed{p = O\left(\frac{T^*(n)}{S(n)}\right)}$$

ดังนั้น **ในบรรดา work-optimal algorithms ตัวที่ step น้อยกว่าจะ scalable กว่า** (คง optimal speedup ได้กับจำนวน processor ที่มากกว่า)

Lower bound ของ step สำหรับปัญหาที่ output ขึ้นกับ input ทั้ง n ตัว และใช้ operator แบบ binary (fan-in 2) เช่น reduce/scan บน EREW/CREW: แต่ละ step รวมข้อมูลได้ อย่างมาก 2 เท่า จึงต้องใช้อย่างน้อย $\lceil \log_2 n \rceil$ step $\Rightarrow S(n) = \Omega(\log n)$. (บน Common CRCW ทำ max ได้ใน $O(1)$ step แต่ต้องใช้ $O(n^2)$ work; cascade แล้วได้ $O(\log \log n)$ step แบบ work-optimal)

ลำดับความสำคัญในการตัดสินว่า WS algorithm ไหน "ดีกว่า"

1. **Work-optimal ก่อน**: $W(n) = \Theta(T^*(n))$
2. **แล้วจึง minimize step**: ให้ $S(n)$ น้อยที่สุด (อุดมคติคือ polylog เช่น $O(\log n)$)
3. ค่าคงที่ (เช่น $\log n$ vs $2 \log n$) ไม่เปลี่ยน asymptotic — ถือว่าเท่ากัน

แนวข้อสอบ

ตัวอย่างการเปรียบเทียบ (สไตล์ Quiz 2)

- A: $W = O(T^*(n))$, $S = O(\sqrt{n})$ vs B: $W = O(T^*(n) \log n)$, $S = O(\log n)$
  $\Rightarrow$ **A ดีกว่า** เพราะ work-optimal: $T_p^A = O(T^*(n)/p + \sqrt{n})$ ให้ optimal speedup ทุก $p = O(T^*(n)/\sqrt{n})$ ส่วน $T_p^B = O(T^*(n) \log n/p + \log n)$ ได้ speedup อย่างมาก $p/\log n$ ไม่มีทาง optimal และแย่ลงเมื่อ n โตขึ้น
- A: $W = O(T^*(n) \log n)$, $S = \log n$ vs B: $W = O(T^*(n))$, $S = 2 \log n$
  $\Rightarrow$ **B ดีกว่า** — work-optimal และ step $O(\log n)$ เท่ากันในเชิง asymptotic

2.6 ลักษณะของ parallel solution ที่ดีบน WS model

1. **Work-optimal** — ไม่ทำงานเกินกว่า sequential ที่ดีที่สุด (มิฉะนั้น speedup หายไปเมื่อ n ใหญ่ และสิ้นเปลืองพลังงาน/ทรัพยากร)
2. **Step น้อย** (polylogarithmic) — parallelism W/S สูง ใช้ processor ได้มาก
3. **ใช้ memory model ที่อ่อนที่สุด** เท่าที่ทำได้ (EREW ดีกว่า CRCW) เพราะ implement ได้จริงง่ายกว่า
4. **อธิบายได้ด้วย pseudocode ที่ชัดเจน** และ map ลง hardware ได้ (เช่น CUDA)

2.7 การเขียน WS pseudocode

หลักการ: เขียนเหมือน sequential แต่ใช้ **for all ... in parallel** กับส่วนที่ขนานกันได้ และใช้ loop ปกติกับส่วนที่ต้องทำเป็นลำดับ (เช่น ระดับของ tree)

Algorithm 1 WS Reduce (balanced tree), $n = 2^k$, operator $\oplus$

Input: $A[1..n]$

Output: $s = A[1] \oplus A[2] \oplus \cdots \oplus A[n]$

1: **for all** $1 \le i \le n$ **in parallel do**
2: $\quad B[i] \leftarrow A[i]$
3: **end for**
4: **for** $h = 1$ **to** $\log n$ **do**
5: $\quad$ **for all** $1 \le i \le n/2^h$ **in parallel do**
6: $\quad\quad B[i] \leftarrow B[2i-1] \oplus B[2i]$
7: $\quad$ **end for**
8: **end for**
9: **return** $B[1]$

วิเคราะห์: $S(n) = 1 + \log n + 1 = O(\log n)$; $W(n) = n + \sum_{h=1}^{\log n} n/2^h + 1 = O(n) \Rightarrow$ work-optimal, EREW.

เทคนิคการวิเคราะห์ด้วย recurrence (สำหรับ recursive/divide-and-conquer):

Recurrence	ผลเฉลย	ตัวอย่าง
$S(n) = S(n/2) + O(1)$	$O(\log n)$	step ของ reduce/scan
$W(n) = 2W(n/2) + O(1)$ หรือ $W(n/2) + O(n)$	$O(n)$	work ของ reduce/scan
$S(n) = S(n/2) + O(\log n)$	$O(\log^2 n)$	merge sort ที่ merge ใช้ $\log n$ step
$W(n) = 2W(n/2) + O(n)$	$O(n \log n)$	merge sort

2.8 เทคนิคการออกแบบที่สำคัญ

Balanced tree สร้าง tree (ในความคิด) บน input แล้ว sweep ขึ้น/ลง — ใช้ใน reduce, scan, broadcast

Pointer jumping ให้แต่ละ node ชี้ไปที่ successor ของ successor (doubling) เช่น หา root ของ forest / list ranking: $S = O(\log h)$, $W = O(n \log h)$ — ไม่ work-optimal

Algorithm cascading (accelerated cascading) รวม algorithm ที่ *เร็วแต่ไม่ work-efficient* กับ algorithm ที่ *ช้าแต่ efficient*: แบ่ง input เป็น block ขนาด $\log n$ แต่ละ block ทำ sequential ($O(\log n)$ step) ได้ผล $n/\log n$ ค่า แล้วค่อยใช้ parallel tree $\Rightarrow$ ลด work ให้เป็น $O(n)$ โดยที่ step ยังเป็น $O(\log n)$ (นี่คือแนวคิดเดียวกับ *thread coarsening* ใน CUDA)

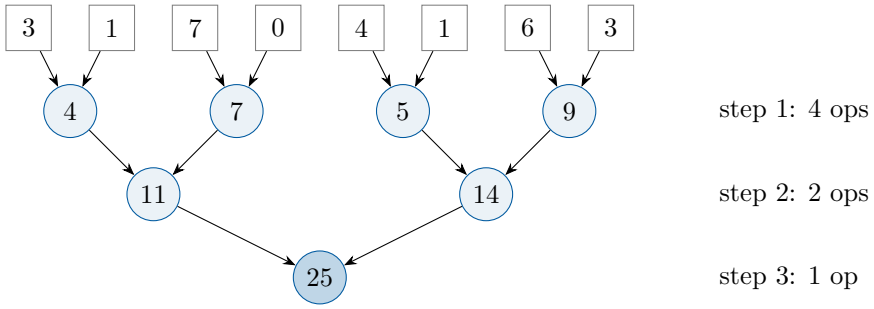
Divide and conquer แบ่งปัญหาย่อย แก้แบบขนาน แล้ว combine (เช่น convex hull, merge sort)

3 Basic Building Blocks: Reduce, Scan, Histogram

3.1 Reduce

Input: $A = \langle a_0, \ldots, a_{n-1} \rangle$ และ binary **associative** operator $\oplus$ (เช่น $+$, $\times$, max, min, AND, OR).
Output: $a_0 \oplus a_1 \oplus \cdots \oplus a_{n-1}$.

- Sequential: $T^*(n) = \Theta(n)$ ($n-1$ operations)
- Parallel (tree): $n-1$ operations ใน $\log n$ step $\Rightarrow W = O(n)$, $S = O(\log n)$
- ต้องการแค่ **associativity** (ลำดับการจับคู่เปลี่ยน แต่ลำดับซ้าย–ขวาไม่เปลี่ยน); ถ้า implementation สลับลำดับด้วย ต้องการ commutativity เพิ่ม
- ระวัง: floating-point $+$ ไม่ associative จริง — ผลอาจต่างจาก sequential เล็กน้อย



3.2 Scan (Prefix Sum)

Input: $\langle a_0, \ldots, a_{n-1} \rangle$, associative operator $\oplus$ และ identity I.

- **Inclusive scan**: $\langle a_0,\ a_0 \oplus a_1,\ \ldots,\ a_0 \oplus \cdots \oplus a_{n-1} \rangle$
- **Exclusive scan**: $\langle I,\ a_0,\ a_0 \oplus a_1,\ \ldots,\ a_0 \oplus \cdots \oplus a_{n-2} \rangle$
- แปลงกันได้: $\text{inclusive}[i] = \text{exclusive}[i] \oplus a_i$; $\text{exclusive}[i] = \text{inclusive}[i-1]$ (shift ขวา)

ตัวอย่าง: $\langle 3,1,7,0,4,1,6,3 \rangle$, $+$, 0
inclusive $= \langle 3,4,11,11,15,16,22,25 \rangle$, exclusive $= \langle 0,3,4,11,11,15,16,22 \rangle$

Sequential scan ใช้ $n-1$ operation ($T^*(n) = \Theta(n)$) จึงต้องการ parallel scan ที่ $W = O(n)$

3.2.1 Hillis–Steele inclusive scan (1986)

Algorithm 2 Hillis–Steele Scan (inclusive)

for $d = 1$ **to** $\log_2 n$ **do**
  for all $0 \le k < n$ **in parallel do**
    if $k \ge 2^{d-1}$ **then**
      $x[k] \leftarrow x[k - 2^{d-1}] \oplus x[k]$
    end if
  end for
end for

- $S(n) = \log n; \quad W(n) = \sum_{d=1}^{\log n}(n - 2^{d-1}) = n \log n - (n-1) = O(n \log n)$
- **ไม่ work-efficient** (sequential ใช้แค่ $n-1$) แต่ step น้อยและโครงสร้างง่าย เหมาะเมื่อ n ไม่ใหญ่กว่าจำนวน thread มาก (เช่น scan ภายใน warp/block)
- ใน implementation จริงต้องใช้ **double buffering** เพราะ semantics ของ forall ต้องอ่านค่าเก่าทั้งหมดก่อนเขียน

3.2.2 Blelloch exclusive scan (1990) — work-efficient

สองเฟส: **up-sweep (reduce phase)** และ **down-sweep**

Algorithm 3 Blelloch Scan (exclusive), $n = 2^k$

for $d = 0$ **to** $\log_2 n - 1$ **do** ▷ up-sweep
  for all $k = 0$ **to** $n - 1$ **by** 2^{d+1} **in parallel do**
    $x[k + 2^{d+1} - 1] \leftarrow x[k + 2^d - 1] \oplus x[k + 2^{d+1} - 1]$
  end for
end for
$x[n-1] \leftarrow I$
for $d = \log_2 n - 1$ **down to** 0 **do** ▷ down-sweep
  for all $k = 0$ **to** $n - 1$ **by** 2^{d+1} **in parallel do**
    $t \leftarrow x[k + 2^d - 1]$
    $x[k + 2^d - 1] \leftarrow x[k + 2^{d+1} - 1]$
    $x[k + 2^{d+1} - 1] \leftarrow t \oplus x[k + 2^{d+1} - 1]$
  end for
end for

- แต่ละเฟส $W = O(n)$ ($n-1$ ops), $S = \log n \Rightarrow$ รวม $W = O(n)$, $S = 2\log n = O(\log n)$
- **work-optimal** — จำนวน operation $\approx 2(n-1)$ คงที่เท่าของ sequential
- Invariant ของ down-sweep: ค่าที่ส่งให้ node คือ "ผลรวมของทุกอย่างที่อยู่ทางซ้ายของ subtree นั้น"
- ใช้ได้กับ operator ที่ไม่ commutative (เช่น function composition) ถ้ารักษาลำดับซ้าย–ขวา

Trace ของตัวอย่าง $\langle 3,1,7,0,4,1,6,3\rangle$:

ขั้น	x
เริ่มต้น	$3,1,7,0,4,1,6,3$
up $d=0$	$3,\mathbf{4},7,\mathbf{7},4,\mathbf{5},6,\mathbf{9}$
up $d=1$	$3,4,7,\mathbf{11},4,5,6,\mathbf{14}$
up $d=2$	$3,4,7,11,4,5,6,\mathbf{25}$
set root $=0$	$3,4,7,11,4,5,6,\mathbf{0}$
down $d=2$	$3,4,7,\mathbf{0},4,5,6,\mathbf{11}$
down $d=1$	$3,\mathbf{0},7,\mathbf{4},4,\mathbf{11},6,\mathbf{16}$
down $d=0$	$\mathbf{0},\mathbf{3},\mathbf{4},\mathbf{11},\mathbf{11},\mathbf{15},\mathbf{16},\mathbf{22}$

3.2.3 Recursive scan (Algorithm 4 ใน `pram.pdf`)

จับคู่ $y_i = x_{2i-1} \oplus x_{2i}$ ($n/2$ ค่า), scan y แบบ recursive ได้ z, แล้ว $s_i = z_{i/2}$ เมื่อ i คู่, $s_1 = x_1$, และ $s_i = z_{(i-1)/2} \oplus x_i$ เมื่อ i คี่ > 1.
$S(n) = S(n/2) + O(1) = O(\log n)$, $W(n) = W(n/2) + O(n) = O(n)$ — work-optimal, EREW.

3.2.4 Primitives ที่สร้างจาก Scan

Primitive	วิธีทำ (W=$O(n)$, S=$O(\log n)$ ทั้งหมด)
Enumerate	flag $\to 0/1$ แล้ว exclusive scan: ได้ลำดับที่ของสมาชิกที่ flag เป็น true
Copy / Broadcast	ใส่ v ที่ตำแหน่งแรก ที่เหลือเป็น identity แล้ว scan
Pack / Filter / Compact	flag $\to$ exclusive scan ได้ index ปลายทาง $\to$ scatter เฉพาะตัวที่ flag true
Split	enumerate(not F) และ enumerate(F) แล้วย้าย false ไปหน้า true ไปหลัง (stable) — แกนของ radix sort
Flatten	scan ขนาดของ sub-array ได้ offset แล้ว copy แบบขนาน
Radix sort	ทำ split ทีละ bit ตั้งแต่ LSB

3.3 Histogram

Input: $A[1..n]$ ที่มีค่าในช่วง $[0,m)$ (หรือ map เข้า bin ด้วย function). **Output**: $H[0..m-1]$ โดย $H[j] = |\{i : A[i] = j\}|$.

- Sequential: `for i: H[A[i]]++` $\Rightarrow T^*(n) = O(n+m)$
- **ความยาก**: thread หลายตัวอาจเขียน bin เดียวกัน (*output conflict*) — ต่างจาก reduce/scan ที่ตำแหน่ง output กำหนดล่วงหน้า
- **Naive parallel (WS)**: สร้าง matrix $R[i][j] = [A[i] = j]$ ขนาด $n \times m$ แล้ว reduce แต่ละ column $\Rightarrow S = O(\log n)$ แต่ $W = O(nm)$ — ไม่ work-optimal
- **Cascaded (Algorithm 6 ใน `pram.pdf`)**: แบ่ง input เป็น n/m block ขนาด m (หรือ p block), แต่ละ block ทำ histogram แบบ sequential ลง *private histogram* ของตัวเอง แล้ว reduce แต่ละ bin

ข้าม block
$\Rightarrow W = O(n)$, $S = O(m + \log n)$ — work-optimal, EREW

- **CRCW / atomic**: ใช้ fetch-and-add (`atomicAdd`) ได้ $W = O(n)$, $S = O(1)$ ในเชิงทฤษฎี แต่ใน hardware จริง การ update bin เดียวกัน *ถูก serialize* (contention)

แนวคิดสำคัญ: Privatization

ให้แต่ละกลุ่มงาน (block) มี **private histogram** ของตัวเอง แล้วค่อย **merge** — เปลี่ยน write conflict แบบ fine-grained (n ครั้ง) ให้เป็น reduce แบบ coarse-grained (#blocks $\times$ m ครั้ง) นี่คือรากฐานของทั้ง Algorithm 6 และ CUDA shared-memory histogram

Histogram + Scan = Counting sort: histogram ของ key $\rightarrow$ exclusive scan ได้ตำแหน่งเริ่มต้นของแต่ละ key $\rightarrow$ วาง element ลงตำแหน่ง (stable ได้ถ้าทำ offset ต่อ block ตาม Figure 13 ใน `parallel.pdf`). $W = O(n + m)$.

3.4 การ Decompose ปัญหาใหม่เป็น Building Blocks

ขั้นตอนคิด

1. **Map**: ข้อมูลแต่ละตัวต้องแปลงอะไรอย่างอิสระ? (flag, ค่าคำนวณย่อย) — $W = O(n)$, $S = O(1)$
2. ต้องการ **คำตอบเดียว** จากข้อมูลทั้งหมด? $\rightarrow$ **Reduce**
3. ต้องการ **คำตอบต่อตำแหน่งที่ขึ้นกับทุกตัวก่อนหน้า** (running total, ตำแหน่งปลายทาง) $\rightarrow$ **Scan**
4. ต้องการ **นับ/รวมตามหมวดหมู่** (key $\rightarrow$ bin) $\rightarrow$ **Histogram**
5. ตรวจว่า operator **associative** หรือไม่ และหา **identity** (บางครั้งต้องนิยาม operator ใหม่บน tuple เช่น (a, b))
6. รวม work/step: ต่อกัน $\Rightarrow$ บวก; ทุก block เป็น $O(n)/O(\log n)$ $\Rightarrow$ รวมได้ $O(n)/O(\log n)$

ปัญหา	Decomposition
นับจำนวนที่มากกว่า t	map: $f_i = [a_i > t] \to$ reduce(+)
ค่าเฉลี่ย / variance	reduce(+) บน a_i และ a_i^2 (หรือ reduce บน tuple $(n, \Sigma x, \Sigma x^2)$)
Index ของตัวแรกที่ $> t$	map: $g_i = i$ ถ้า $a_i > t$ มิฉะนั้น $\infty \to$ reduce(min)
ลบศูนย์ออกจาก array (compact)	map flag $\to$ exclusive scan(+) $\to$ scatter
Running maximum	inclusive scan(max)
Line of sight	map: มุม $\theta_i \to$ exclusive scan(max) $\to$ map: มองเห็นถ้า $\theta_i > \max_{j<i} \theta_j$
วงเล็บสมดุล	map: ($\to +1$,) $\to -1 \to$ inclusive scan(+) $\to$ reduce(min)≥ 0 และตัวสุดท้าย $= 0$
Polynomial $\sum a_i x^i$	map: ทุกตัวเป็น $x \to$ exclusive scan($\times$, 1) ได้ $x^i \to$ map $a_i x^i \to$ reduce(+)
Linear recurrence $x_i = a_i x_{i-1} + b_i$	scan บนคู่ (a, b) ด้วย $(a_1, b_1) \otimes (a_2, b_2) = (a_1 a_2,\ a_2 b_1 + b_2)$ (function composition)
Counting sort	histogram $\to$ exclusive scan $\to$ scatter
Median / percentile ของข้อมูลจำนวนเต็ม	histogram $\to$ inclusive scan $\to$ หา bin แรกที่ prefix $\geq n/2$ (map + reduce min)
Run-length encoding	map: head flag $[a_i \neq a_{i-1}] \to$ exclusive scan ได้ index $\to$ scatter ค่าและตำแหน่ง $\to$ map: length = ตำแหน่งถัดไป $-$ ตำแหน่งนี้
Lexical analysis (DFA)	map สัญลักษณ์เป็น transition function $\to$ scan ด้วย composition (non-commutative)

4 Algorithms ที่สร้างจาก Building Blocks: Filter, Flatten, Search, Merge, K-th Smallest

หัวข้อนี้อ้างอิง Section 4.2–4.5 ของ `parallel.pdf` ทุก algorithm ในหัวข้อนี้ประกอบจาก map, scan, reduce และ divide-and-conquer จึงเป็นตัวอย่างที่ดีของการ *decompose* ปัญหาใหม่

4.1 Filter (Pack / Stream compaction)

Input: sequence A และ predicate p. **Output**: สมาชิก $a \in A$ ที่ $p(a)$ เป็นจริง **เรียงตามลำดับเดิม** (stable)

Algorithm 4 filter(A, p) (Figure 3 ใน `parallel.pdf`)

1: $n \leftarrow |A|$
2: **for all** $0 \leq i < n$ **in parallel do**
3: $\quad F[i] \leftarrow p(A[i])\ ?\ 1 : 0$ ▷ map: flag
4: **end for**
5: $(X, m) \leftarrow \text{plusScan}(F)$ ▷ exclusive scan + total
6: $R \leftarrow$ array ขนาด m
7: **for all** $0 \leq i < n$ **in parallel do**
8: $\quad$ **if** $F[i] = 1$ **then**
9: $\quad\quad R[X[i]] \leftarrow A[i]$ ▷ scatter
10: $\quad$ **end if**
11: **end for**
12: **return** R

วิเคราะห์: map และ scatter ใช้ $W = O(n)$, $S = O(1)$; scan ใช้ $W = O(n)$, $S = O(\log n)$ $\Rightarrow$ $\boxed{W = O(n),\ S = O(\log n)}$ work-optimal และใช้ EREW ได้ (index $X[i]$ ของตัวที่ผ่านไม่ซ้ำกัน)

ตัวอย่าง: $A = \langle 4, 9, 1, 7, 3, 8, 2, 6\rangle$, $p(x) = [x > 5]$
$F = \langle 0, 1, 0, 1, 0, 1, 0, 1\rangle$, $X = \langle 0, 0, 1, 1, 2, 2, 3, 3\rangle$, $m = 4 \Rightarrow R = \langle 9, 7, 8, 6\rangle$

บน CUDA filter เป็น memory-bound (อ่าน n เขียน m) จึงวัดด้วย GB/s:

- ไม่ต้องเขียน array F และ X ขนาด n ลง global: คำนวณ flag ใน register แล้ว scan ภายใน block (kernel fusion)
- ใน warp ใช้ `__ballot_sync` ได้ flag 32 ตัวใน word เดียว แล้วใช้ `__popc(mask & lanemask_lt)` เป็น exclusive scan
- offset ระหว่าง block: scan ของจำนวนต่อ block (หลาย kernel) หรือ decoupled look-back (single pass)
- ถ้าไม่ต้องคงลำดับ ใช้ `atomicAdd` หนึ่งครั้งต่อ warp เพื่อจองที่ใน output ได้ (เร็วกว่า แต่ **ไม่ stable**)

4.2 Flatten

Input: nested sequence A (sequence ของ sequence). **Output**: sequence เดียวที่นำ $A[0], A[1], \ldots$ มาต่อกัน
เช่น flatten$(\langle\langle 3, 1, 2\rangle, \langle 5, 1\rangle, \langle 7, 8\rangle\rangle) = \langle 3, 1, 2, 5, 1, 7, 8\rangle$

Algorithm 5 flatten(A) (Figure 4 ใน `parallel.pdf`)

1: **for all** $0 \le i < |A|$ **in parallel do**
2: $\quad sizes[i] \leftarrow |A[i]|$
3: **end for**
4: $(X, m) \leftarrow$ plusScan$(sizes)$ ▷ offset ของแต่ละ sub-array
5: $R \leftarrow$ array ขนาด m
6: **for all** $0 \le i < |A|$ **in parallel do**
7: $\quad$ **for all** $0 \le j < |A[i]|$ **in parallel do**
8: $\quad\quad R[X[i] + j] \leftarrow A[i][j]$
9: $\quad$ **end for**
10: **end for**
11: **return** R

ให้ $n = |A|$ (จำนวน sub-array) และ $m =$ ขนาด output: $\boxed{W = O(n + m),\ S = O(\log n)}$ work-optimal (sequential ต้องใช้ $\Theta(n + m)$ เช่นกัน)

ตัวอย่าง: sizes $= \langle 3, 2, 2\rangle \to X = \langle 0, 3, 5\rangle$, $m = 7$ — $A[1] = \langle 5, 1\rangle$ ถูก copy ไปที่ $R[3..4]$

บน CUDA: ปัญหา load imbalance ถ้าใช้ 1 thread ต่อ sub-array แล้วขนาดของ sub-array ต่างกันมาก (เช่น 1 กับ 10,000) thread ส่วนใหญ่จะรอ thread ที่ทำงานเยอะ และการเขียนจะไม่ coalesced. วิธีที่ดีกว่าคือใช้ **1 thread ต่อ 1 output element** j:

- หาว่า j อยู่ใน sub-array ไหน: $i = \texttt{upper_bound}(X, j) - 1$ (binary search ใน X, ข้าม sub-array ว่างได้เอง) หรือใช้ scan: วาง i ที่ตำแหน่ง $X[i]$ (เฉพาะ sub-array ที่ไม่ว่าง) แล้วทำ inclusive scan(max)
- แล้ว $R[j] \leftarrow A[i][\,j - X[i]\,]$ — thread ติดกันเขียน R ติดกัน (coalesced) และงานเท่ากันทุก thread

แนวคิดเดียวกันใช้กับงาน "expand" ทุกแบบ เช่น สร้าง edge list จาก adjacency list (CSR) ใน BFS

4.3 Search (Sorted search)

ปัญหา: กำหนด sorted sequence A และ key v หาตำแหน่งของสมาชิกที่มากที่สุดใน A ที่น้อยกว่า v
Binary search: $W = S = O(\log n)$ — ทำงานแบบ sequential ได้ดีที่สุดแล้ว แต่ step ยังเป็น $O(\log n)$

k-way search (แลก work เพื่อลด step): แบ่ง A เป็น k block ขนาด $s = |A|/k$ ใช้ k processor ตรวจพร้อมกันว่า v อยู่ใน block ไหน แล้วค้นต่อเฉพาะใน block นั้น

Algorithm 6 search(A, v, k) (Figure 5 ใน `parallel.pdf`)

```
1:  function findBlock(A, v, k)
2:      s ← |A|/k
3:      for all 0 ≤ i < k in parallel do
4:          if A[i · s] < v and A[(i + 1) · s] > v then
5:              r ← i
6:          end if                                   ▷ A[|A|] ถือเป็น +∞
7:      end for
8:      return (A[rs : (r + 1)s], rs)
9:  end function
10: function search(A, v, k)
11:     (B, o) ← findBlock(A, v, min(|A|, k))
12:     if |A| ≤ k then return o
13:     else return o+search(B, v, k)
14:     end if
15: end function
```

- แต่ละรอบลดขนาดลง k เท่า ⇒ มี $\log_k n$ รอบ, รอบละ $O(1)$ step และ $O(k)$ work
- เลือก $k = n^\epsilon$: $S = O(1/\epsilon)$, $W = O(n^\epsilon/\epsilon)$. เช่น $\epsilon = 1/2$: $O(1)$ **step** แต่ work $O(\sqrt{n})$ (มากกว่า binary search $O(\log n)$ มาก)
- เลือก $k = 2$ ก็กลับมาเป็น binary search ($W = S = O(\log n)$)
- มีเพียง processor เดียวที่เขียน r (ถ้า v ไม่ซ้ำกับสมาชิก) แต่ทุกตัวอ่าน v พร้อมกัน ⇒ CREW

ตัวอย่าง: $A = \langle 2, 4, 6, \ldots, 32\rangle$ $(n = 16)$, $v = 19$, $k = 4$
รอบ 1: $s = 4$ เทียบ $A[0], A[4], A[8], A[12] = 2, 10, 18, 26$ ⇒ $18 < 19 < 26$ ได้ block 2 ($A[8..11] = \langle 18, 20, 22, 24\rangle$, $o = 8$)
รอบ 2: $s = 1$ ได้ $18 < 19 < 20$ ⇒ $r = 0$ ⇒ ตอบตำแหน่ง $8 + 0 = 8$ (ค่า 18)

ค้นหาหลาย key พร้อมกัน ในทางปฏิบัติมักต้องค้น m key ใน A เดียวกัน (เช่น ใน merge หรือ flatten): ทำ binary search ทุก key แบบขนาน ได้ $W = O(m \log n)$, $S = O(\log n)$ — บน GPU แต่ละ thread ทำ binary search ของตัวเอง (โหลดระดับบน ๆ ของ A ไว้ใน shared memory เพื่อลด global access)

4.4 K-th of two sorted arrays (`kth`, co-rank)

ปัญหา: กำหนด sorted A, B และ k หา (l_a, l_b) ที่ $l_a + l_b = k$ และทุกตัวใน $A[0 : l_a] \cup B[0 : l_b]$ น้อยกว่าหรือเท่ากับทุกตัวใน $A[l_a :] \cup B[l_b :]$ (คือ "k ตัวแรกของผล merge มาจาก A กี่ตัว จาก B กี่ตัว")

Dual binary search (Figure 6): เทียบ $A[|A|/2]$ กับ $B[|B|/2]$ แล้วตัดครึ่งหนึ่งของ A หรือ B ทิ้งทุกครั้ง $\Rightarrow W = S = O(\log |A| + \log |B|)$

รูปแบบที่ใช้บ่อยบน GPU (co-rank / merge path): binary search หา $l_a = i$ ในช่วง $[\max(0, k - |B|),\ \min(k, |A|)]$ ให้

$$A[i-1] \le B[k-i] \qquad \text{และ} \qquad B[k-i-1] < A[i]$$

(ค่านอกขอบถือเป็น $-\infty/+\infty$; ใช้ $\le$ กับ $<$ แบบนี้เพื่อให้ A มาก่อนเมื่อค่าเท่ากัน คือ stable) — $W = S = O(\log \min(k, |A|))$

ตัวอย่าง: $A = \langle 1, 4, 6, 9 \rangle$, $B = \langle 2, 3, 7, 8 \rangle$
$k = 4$: 4 ตัวแรกคือ $1, 2, 3, 4 \Rightarrow (l_a, l_b) = (2, 2)$; ตรวจ $A[1] = 4 \le B[2] = 7$ และ $B[1] = 3 < A[2] = 6$ ✓
$k = 5$: 5 ตัวแรกคือ $1, 2, 3, 4, 6 \Rightarrow (3, 2)$; ตรวจ $A[2] = 6 \le B[2] = 7$ และ $B[1] = 3 < A[3] = 9$ ✓

4.5 Merge

ปัญหา: รวม sorted A และ B (รวม $m = |A| + |B|$ ตัว) เป็น sorted R แบบ stable. Sequential merge ใช้ $\Theta(m)$ แต่เป็นลำดับทั้งหมด (step $= m$)

(1) Merge ด้วย rank (ง่าย แต่ไม่ work-optimal)

ตำแหน่งของ $a = A[i]$ ใน R คือ $i + \text{rank}_B(a)$ โดย rank คือจำนวนตัวใน B ที่ *น้อยกว่า* a; ตำแหน่งของ $b = B[j]$ คือ $j + \text{rank}_A(b)$ โดยนับตัวใน A ที่ *น้อยกว่าหรือเท่ากับ* b (ใช้ $<$ กับ $\le$ ต่างกันเพื่อไม่ให้ตำแหน่งชนกันเมื่อค่าเท่ากัน และให้ A มาก่อน)
ทุกตัวทำ binary search พร้อมกัน: $W = O(m \log m)$, $S = O(\log m)$ **ไม่ work-optimal**

ตัวอย่าง: $A = \langle 1, 4, 6, 9 \rangle$, $B = \langle 2, 3, 7, 8 \rangle$
A: $1 \to 0 + 0 = 0$, $4 \to 1 + 2 = 3$, $6 \to 2 + 2 = 4$, $9 \to 3 + 4 = 7$; B: $2 \to 0 + 1 = 1$, $3 \to 1 + 1 = 2$, $7 \to 2 + 3 = 5$, $8 \to 3 + 3 = 6$
$\Rightarrow R = \langle 1, 2, 3, 4, 6, 7, 8, 9 \rangle$

(2) Divide-and-conquer merge ด้วย `kth` (Figure 7)

Algorithm 7 merge(A, B, R)

if $|A| = 0$ **then** copy B ไป R; **return**
end if
if $|B| = 0$ **then** copy A ไป R; **return**
end if
$h \leftarrow |R|/2$
$(m_a, m_b) \leftarrow \text{kth}(A, B, h)$ ▷ แบ่ง output ครึ่งพอดี
in parallel do
  merge$(A[0 : m_a],\ B[0 : m_b],\ R[0 : h])$
  merge$(A[m_a :],\ B[m_b :],\ R[h :])$

ใช้ dual binary search สำหรับ kth:

$$W(m) = 2W(m/2) + O(\log m) = O(m), \qquad S(m) = S(m/2) + O(\log m) = O(\log^2 m)$$

work-optimal แล้ว. ถ้าใช้ kth แบบขนาน (k-way) จะได้ $S = O(\log m)$; ถ้าแบ่งเป็น $\sqrt{m}$ ส่วน (Figure 8) บน MP-RAM ได้ถึง $S = O(\log \log m)$ (Bound 4.1: merge ใช้ $O(n)$ work, $O(\log n)$ span บน binary-forking)

(3) แบ่ง output เป็น p ส่วนเท่ากัน + sequential merge (merge path) — แบบที่ใช้จริง

1. `forall` $t < p$: $(a_t, b_t) \leftarrow \text{kth}(A, B,\ t \cdot m/p)$ — p binary search, $O(\log m)$ step
2. `forall` $t < p$: merge $A[a_t : a_{t+1}]$ กับ $B[b_t : b_{t+1}]$ แบบ sequential ลง $R[tm/p : (t+1)m/p]$ — $O(m/p)$ step

$W = O(m + p \log m)$, $S = O(m/p + \log m)$. เลือก $p = m/\log m$ ได้ $\boxed{W = O(m),\ S = O(\log m)}$ (นี่คือ algorithm cascading อีกแบบ)

บน CUDA: แต่ละ thread ได้ output เท่ากันพอดี (เช่น 8–16 ตัว) จึง **load balance** ดีไม่ว่าข้อมูลจะกระจายอย่างไร; ปรับจูนโดยให้ block หา co-rank ของ tile ก่อน แล้วโหลด tile ของ A, B เข้า shared memory แบบ coalesced ให้ thread merge จาก shared memory แล้วเขียน output ผ่าน shared memory เพื่อให้เขียน global แบบ coalesced

4.6 K-th smallest (Selection)

ปัญหา: หาสมาชิกตัวที่ k (นับจาก 0) เมื่อเรียง A จากน้อยไปมาก — โดยไม่ต้อง sort ทั้งหมด ($k = \lfloor n/2 \rfloor$ คือ median)

Algorithm 8 kthSmallest(A, k) (Figure 9 ใน `parallel.pdf`)

1: $p \leftarrow \text{selectPivot}(A)$
2: $L \leftarrow \text{filter}(A,\ x < p)$; $\quad G \leftarrow \text{filter}(A,\ x > p)$ ▷ ทำพร้อมกันได้
3: **if** $k < |L|$ **then return** kthSmallest(L, k)
4: **else if** $k \geq |A| - |G|$ **then return** kthSmallest$(G,\ k - (|A| - |G|))$
5: **else return** p ▷ ตัวที่เท่ากับ p อยู่ช่วงกลาง
6: **end if**

- **Random pivot**: มีโอกาส $1/2$ ที่ pivot อยู่ช่วงกลาง (quartile ที่ 1–3) ทำให้ขนาดลดเหลือ $\leq \frac{3}{4} \Rightarrow$ เรียก recursive $O(\log n)$ ครั้ง w.h.p.; แต่ละครั้งใช้ filter ($S = O(\log n)$)
  $\Rightarrow \boxed{W = O(n) \text{ (expected)},\ S = O(\log^2 n) \text{ w.h.p.}}$ (work เป็นอนุกรมเรขาคณิต $n + \frac{3}{4}n + \cdots$)
- **Worst case** (pivot แย่ทุกครั้ง เช่น เลือกตัวน้อยสุด): $W = O(n^2)$
- **Deterministic (median of medians, block ละ 5)**: $W(n) = W(\frac{7}{10}n) + W(\frac{1}{5}n) + O(n) = O(n)$ แต่ $S = O(n^\alpha)$, $\alpha \approx 0.84$ (ขนานได้ไม่ดี)
- เทียบกับ sort ก่อนแล้วหยิบตัวที่ k: $W = O(n \log n)$ **ไม่ work-optimal**

ตัวอย่าง: $A = \langle 7, 2, 9, 4, 3, 8, 1, 6 \rangle$, $k = 3$ (ตัวที่ 4 จากน้อย; sorted คือ $1, 2, 3, 4, 6, 7, 8, 9$ ตอบ 4) ใช้ pivot = ตัวแรก

รอบ	A	p	L	G	การตัดสิน
1	$7, 2, 9, 4, 3, 8, 1, 6$	7	$2, 4, 3, 1, 6$	$9, 8$	$k = 3 < \|L\| = 5 \rightarrow$ ไป L, $k = 3$
2	$2, 4, 3, 1, 6$	2	1	$4, 3, 6$	$k = 3 \geq 5 - 3 = 2 \rightarrow$ ไป G, $k = 3 - 2 = 1$
3	$4, 3, 6$	4	3	6	$k = 1$: ไม่ < 1 และไม่ $\geq 3 - 1 = 2 \rightarrow$ **ตอบ 4**

Selection ด้วย histogram (radix select) — เหมาะกับ GPU สำหรับจำนวนเต็ม 32-bit: (1) histogram ของ 8 bit บนสุด (256 bins) $\rightarrow$ (2) exclusive scan ของ histogram $\rightarrow$ (3) หา bucket d ที่มีตำแหน่ง k (prefix$[d] \leq k <$ prefix$[d+1]$) $\rightarrow$ (4) filter เฉพาะตัวใน bucket d และปรับ $k \leftarrow k -$ prefix$[d] \rightarrow$ ทำซ้ำกับ 8 bit ถัดไป (ไม่เกิน 4 รอบ). ทุกขั้นเป็น building block (histogram, scan, filter) และ $W = O(n)$; รอบแรกอ่านข้อมูลทั้งหมด รอบถัดไปเหลือข้อมูลน้อยลงมาก (ถ้ากระจายสม่ำเสมอ เหลือประมาณ $1/256$ ต่อรอบ) จึงใช้ memory traffic น้อยกว่า radix sort ทั้งชุด

4.7 สรุป

ปัญหา	W	S	Building blocks
Filter	$O(n)$	$O(\log n)$	map → scan → scatter
Flatten	$O(n+m)$	$O(\log n)$	map (sizes) → scan (offsets) → copy
Search ($k = n^\epsilon$)	$O(n^\epsilon/\epsilon)$	$O(1/\epsilon)$	k-way compare (CREW); $k=2$ คือ binary search
kth (2 sorted)	$O(\log m)$	$O(\log m)$	dual binary search / co-rank
Merge (rank)	$O(m \log m)$	$O(\log m)$	binary search ทุกตัว → scatter (ไม่ optimal)
Merge (D&C, kth)	$O(m)$	$O(\log^2 m)$	kth → recurse 2 ด้าน
Merge ($p = m/\log m$)	$O(m)$	$O(\log m)$	kth หลายจุด → sequential merge (cascading)
K-th smallest (random)	$O(n)$ exp.	$O(\log^2 n)$ whp	filter ×2 → recurse
K-th smallest (radix)	$O(n)$	$O(\log n)$ ต่อรอบ	histogram → scan → filter

5 จาก High-level Idea สู่ Low-level CUDA Implementation

5.1 CUDA Programming Model โดยย่อ

- **Hierarchy**: grid → block → thread; thread ถูกจัดเป็น **warp** ละ 32 ตัว ที่รันคำสั่งเดียวกันพร้อมกัน (SIMT)
- Block ถูก schedule ลงบน **SM** (Streaming Multiprocessor); block หนึ่งอยู่บน SM เดียวตลอด
- **Synchronization**: `__syncthreads()` ใช้ได้ *ภายใน block เท่านั้น*; การ sync ข้าม block ทั้ง grid ทำได้โดย **จบ kernel แล้ว launch kernel ใหม่**
- **Atomic operations**: `atomicAdd`, `atomicCAS`, `atomicMax` ฯลฯ (คล้าย fetch-and-add / CAS ใน MP-RAM)

Memory	Scope	ความเร็ว	หมายเหตุ
Register	thread	เร็วที่สุด	จำนวนจำกัดต่อ SM (มีผลต่อ occupancy)
Shared memory	block	เร็ว (on-chip)	แบ่งเป็น 32 **banks**; ระวัง bank conflict
L1 / L2 cache	SM / ทั้ง GPU	กลาง	
Global memory (DRAM)	ทั้ง grid	ช้า (latency หลายร้อย cycle)	ต้อง **coalesced access**
Constant memory	ทั้ง grid (read-only)	เร็วเมื่อ broadcast	

5.2 การ Map แนวคิด WS/PRAM ลง CUDA

High-level (WS / PRAM)	Low-level (CUDA)
`for all` i `in parallel`	thread หนึ่งตัวต่อ i: `i = blockIdx.x*blockDim.x + threadIdx.x`
แต่ละ parallel step (ต้องรอทุกตัวเสร็จ)	`__syncthreads()` (ภายใน block) หรือ kernel launch ใหม่ (ทั้ง grid)
Synchronous semantics (อ่านก่อนเขียน)	double buffer หรือ อ่านลง register → sync → เขียน → sync
p processors (Brent)	จำนวน thread ที่ active พร้อมกันจริง ≈ #SM × threads/SM
Algorithm cascading	**Thread coarsening**: thread หนึ่งทำหลาย element แบบ sequential ก่อน แล้วค่อย tree
Privatization (histogram)	histogram ใน shared memory ต่อ block แล้ว `atomicAdd` ลง global
Concurrent write (CRCW)	atomic operations (ซึ่งถูก serialize เมื่อชนกัน)
Hierarchical decomposition	warp-level → block-level → grid-level

5.3 Reduce บน CUDA

5.3.1 Practice 4.1 — ใช้ global memory อย่างเดียว

แต่ละ step ของ tree = หนึ่ง kernel launch (เพราะต้อง sync ข้าม block)

```
__global__ void reduceStep(float *x, int len, int half) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < len / 2)                // sequential addressing
        x[i] += x[i + half];
}
// host: log n launches
for (int len = n; len > 1; len = (len + 1) / 2) {
    int half = (len + 1) / 2;
    int threads = 256, blocks = (len / 2 + threads - 1) / threads;
    reduceStep<<<blocks, threads>>>(d_x, len, half);
}
// result in d_x[0]
```

ข้อเสีย: อ่าน/เขียน global memory ทุก step, launch overhead $\log n$ ครั้ง

5.3.2 Practice 4.2 — per-block shared memory

แต่ละ block reduce ส่วนของตัวเองใน shared memory แล้วเขียนผลบางส่วน (partial sum) ออกมา จากนั้น reduce partial sums อีกรอบ (launch ใหม่ หรือ `atomicAdd`)

```
__global__ void reduceShared(const float *in, float *out, int n) {
    extern __shared__ float sdata[];
    unsigned tid = threadIdx.x;
    unsigned i = blockIdx.x * (blockDim.x * 2) + tid;
    float v = 0.0f;                                  // first add during load
```

```
    if (i < n)              v  = in[i];
    if (i + blockDim.x < n) v += in[i + blockDim.x];
    sdata[tid] = v;
    __syncthreads();
    for (unsigned s = blockDim.x / 2; s > 0; s >>= 1) {  // sequential addressing
        if (tid < s) sdata[tid] += sdata[tid + s];
        __syncthreads();
    }
    if (tid == 0) out[blockIdx.x] = sdata[0];
}
```

5.3.3 ขั้นตอนการ fine-tune reduce (ตาม principle)

Version	ปัญหาที่แก้	Principle
1. Interleaved: `if (tid % (2*s) == 0)`	— (baseline)	warp divergence สูง, `%` ช้า
2. Interleaved + strided index `idx = 2*s*tid`	ลด divergence	แต่เกิด **shared memory bank conflict**
3. Sequential addressing `tid < s; sdata[tid+s]`	ไม่มี bank conflict, thread ที่ active ติดกัน	ใช้ bandwidth ของ shared memory เต็ม
4. First add during global load	ครึ่งหนึ่งของ thread ว่างตั้งแต่ step แรก	ลดจำนวน block ครึ่งหนึ่ง เพิ่ม work ต่อ thread
5. Unroll last warp / warp shuffle	ใน 32 thread สุดท้ายไม่ต้อง `__syncthreads`	ลด instruction overhead
6. Complete unroll (template block size)	ตัด loop overhead	instruction throughput
7. Multiple elements per thread (grid-stride loop)	ลด overhead ของ tree ต่อ element	**Algorithm cascading / Brent**: work-optimal และ memory-bound

```
__inline__ __device__ float warpReduceSum(float v) {
    for (int off = 16; off > 0; off >>= 1)
        v += __shfl_down_sync(0xffffffff, v, off);   // register-to-register
    return v;
}
__global__ void reduceFinal(const float *in, float *out, int n) {
    __shared__ float warpSums[32];
    float v = 0.0f;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
         i += blockDim.x * gridDim.x)                  // coalesced + coarsened
        v += in[i];
    v = warpReduceSum(v);
    int lane = threadIdx.x & 31, wid = threadIdx.x >> 5;
    if (lane == 0) warpSums[wid] = v;
    __syncthreads();
    if (wid == 0) {
        v = (lane < (blockDim.x >> 5)) ? warpSums[lane] : 0.0f;
```

```
        v = warpReduceSum(v);
        if (lane == 0) atomicAdd(out, v);            // one atomic per block
    }
}
```

เลือก `gridDim` ≈ (#SM × จำนวน block ที่ resident ต่อ SM) ก็พอ — ตรงกับผล Brent: ใช้ p ไม่เกินที่จำเป็น แต่ละ "processor" ทำ n/p element แบบ sequential ก่อน

5.4 Scan บน CUDA

5.4.1 Hillis–Steele (single block, double buffer)

```
__global__ void scanHS(const float *in, float *out, int n) {  // n == blockDim.x
    extern __shared__ float temp[];                            // size 2*n
    int tid = threadIdx.x, pout = 0, pin = 1;
    temp[tid] = in[tid];
    __syncthreads();
    for (int offset = 1; offset < n; offset <<= 1) {
        pout = 1 - pout; pin = 1 - pout;                       // swap buffers
        if (tid >= offset)
            temp[pout*n + tid] = temp[pin*n + tid] + temp[pin*n + tid - offset];
        else
            temp[pout*n + tid] = temp[pin*n + tid];
        __syncthreads();
    }
    out[tid] = temp[pout*n + tid];
}
```

ถ้าเขียน in-place (`temp[tid] += temp[tid-offset]`) จะเกิด **race condition**: thread หนึ่งอาจอ่านค่าที่ thread อื่นเพิ่งเขียนใน step เดียวกัน ซึ่งผิด semantics ของ forall

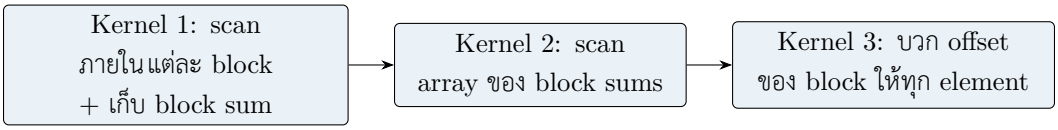
5.4.2 Blelloch (single block, $n = 2\times$`blockDim`)

```
__global__ void scanBlelloch(const float *in, float *out, int n) {
    extern __shared__ float temp[];
    int tid = threadIdx.x, offset = 1;
    temp[2*tid] = in[2*tid]; temp[2*tid+1] = in[2*tid+1];
    for (int d = n >> 1; d > 0; d >>= 1) {          // up-sweep
        __syncthreads();
        if (tid < d) {
            int ai = offset*(2*tid+1)-1, bi = offset*(2*tid+2)-1;
            temp[bi] += temp[ai];
        }
        offset <<= 1;
    }
    if (tid == 0) temp[n-1] = 0;                    // identity at root
```

```
    for (int d = 1; d < n; d <<= 1) {                // down-sweep
        offset >>= 1;
        __syncthreads();
        if (tid < d) {
            int ai = offset*(2*tid+1)-1, bi = offset*(2*tid+2)-1;
            float t = temp[ai]; temp[ai] = temp[bi]; temp[bi] += t;
        }
    }
    __syncthreads();
    out[2*tid] = temp[2*tid]; out[2*tid+1] = temp[2*tid+1];
}
```

Fine-tune: index ที่เป็น stride 2^d ทำให้เกิด **bank conflict** ใน shared memory แก้ด้วย **padding**: เพิ่ม offset `(idx >> 5)` ให้ทุก index (เว้น 1 ช่องทุก 32 ช่อง)

5.4.3 Scan ขนาดใหญ่ (หลาย block) — hierarchical scan



$W = O(n)$ ยังคง work-optimal; ถ้าจำนวน block มากเกิน 1 block ใน kernel 2 ให้ทำ recursive. ทางเลือกที่เร็วกว่า: single-pass scan (decoupled look-back) ลดการอ่าน global memory จาก $\sim 4n$ เหลือ $\sim 2n$.

5.5 Histogram บน CUDA

5.5.1 Naive: global atomics

```
__global__ void histGlobal(const unsigned char *in, unsigned *bins, size_t n) {
    for (size_t i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
         i += (size_t)blockDim.x * gridDim.x)
        atomicAdd(&bins[in[i]], 1u);
}
```

ปัญหา: atomic บน global memory มี latency สูง และเมื่อหลาย thread update bin เดียวกัน จะ **serialize** (contention) — throughput ลดลงมากเมื่อข้อมูลกระจุกตัว (เช่น ตัวอักษรภาษาอังกฤษ)

5.5.2 Practice 7.2 — Privatized histogram ใน shared memory

```
#define NUM_BINS 128
__global__ void histPrivate(const unsigned char *in, unsigned *bins, size_t n) {
    __shared__ unsigned sh[NUM_BINS];
    for (int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) sh[b] = 0;
    __syncthreads();
    for (size_t i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
```

```
        i += (size_t)blockDim.x * gridDim.x)
        atomicAdd(&sh[in[i] & 0x7F], 1u);            // fast shared-memory atomic
    __syncthreads();
    for (int b = threadIdx.x; b < NUM_BINS; b += blockDim.x)
        if (sh[b]) atomicAdd(&bins[b], sh[b]);       // merge: NUM_BINS atomics/block
}
```

จำนวน global atomic ลดจาก n เหลือ $\leq$ #blocks $\times$ `NUM_BINS`

5.5.3 Practice 7.1 — 4096 bins, saturate ที่ 127

- Private histogram 4096 bins $\times$ 4 bytes = 16 KB ต่อ block — ใส่ shared memory ได้ (แต่ลด occupancy: block ต่อ SM ถูกจำกัดด้วย shared memory)
- **กับดัก**: input ยาวได้ถึง 2^{32} ตัว ถ้าทุกตัวเป็นค่าเดียวกัน counter 32-bit จะ **overflow วนกลับเป็น 0** ก่อนถึงขั้น clamp $\Rightarrow$ ต้อง saturate ระหว่างทาง (เช่น `if (sh[v] < 127) atomicAdd(&sh[v],1)`) และใช้ `size_t` เป็น index
- ขั้นสุดท้าย (kernel ที่สองหรือท้าย kernel): `bins[b] = min(bins[b], 127)`

5.5.4 เทคนิค fine-tune histogram เพิ่มเติม

- **Interleaved (coalesced) partitioning**: thread ติดกันอ่าน element ติดกัน (grid-stride loop) ดีกว่าให้แต่ละ thread อ่าน chunk ต่อเนื่องของตัวเอง
- **Aggregation**: ถ้าข้อมูลมีค่าซ้ำติดกันบ่อย ให้ thread สะสมค่าซ้ำใน register ก่อน แล้วค่อย atomic ครั้งเดียว — ลด contention
- **Per-warp sub-histogram**: ลด contention ภายใน block เพิ่มอีกชั้น (แลกกับ shared memory)
- **Coarsening**: จำนวน block น้อยลง $\Rightarrow$ merge น้อยลง

6 หลักการ Fine-tune: วัดได้, Maximize Throughput, Utilize Bandwidth

6.1 หลักการต้องวัดได้ (measurable)

การปรับแต่งต้องอ้างอิงตัวเลขที่วัดได้ ไม่ใช่ความรู้สึก

ตัวชี้วัดหลัก

- **Execution time**: วัดด้วย `cudaEvent` (ไม่รวม/รวม memcpy ให้ชัดเจน, warm-up, เฉลี่ยหลายรอบ)
- **Effective bandwidth**: $BW_{\text{eff}} = \dfrac{\text{bytes read} + \text{bytes written}}{\text{time}}$ เทียบกับ **peak bandwidth** ของ GPU $\Rightarrow$ % of peak

- **Throughput**: elements/s หรือ GFLOP/s เทียบกับ peak compute
- **Arithmetic intensity** (AI) $= \dfrac{\text{FLOPs}}{\text{bytes}}$ — ใช้ **roofline**: performance $\leq \min(\text{peak FLOPs},\ AI \times \text{peak BW})$
- **Occupancy** $= \dfrac{\text{active warps ต่อ SM}}{\text{max warps ต่อ SM}}$
- Profiler (Nsight Compute): memory throughput, achieved occupancy, warp divergence, bank conflicts, atomic throughput

ตัวอย่างการคำนวณ Reduce $n = 2^{24}$ float ใช้เวลา 0.5 ms: bytes $= 2^{24} \times 4 = 67{,}108{,}864$ B $\Rightarrow BW_{\text{eff}} =$ 67.1 MB/0.5 ms $\approx$ 134 GB/s. ถ้า peak = 900 GB/s ได้เพียง $\approx 15\%$ — reduce มี AI = 1 FLOP/4 B = 0.25 จึงเป็น **memory-bound** เป้าหมายคือ เข้าใกล้ peak bandwidth (ไม่ใช่ peak FLOPs)

6.2 Maximize throughput

- **Work-efficient algorithm ก่อน**: เลือก Blelloch แทน Hillis–Steele เมื่อ n ใหญ่; ใช้ coarsening ให้ work รวมใกล้ sequential
- **Latency hiding**: มี warp พร้อมรันมากพอ (occupancy) และ/หรือ ILP (แต่ละ thread โหลดหลายค่าก่อนใช้). Little's law: ข้อมูลที่ต้องอยู่ระหว่างทาง = latency × bandwidth
- **ลด warp divergence**: ให้ thread ที่ทำงานอยู่ติดกัน (sequential addressing) เพื่อให้ทั้ง warp idle พร้อมกันแทนที่จะ idle ครึ่ง ๆ
- **ลด synchronization และ atomic contention**: warp shuffle, privatization, ลดจำนวน kernel launch
- **ลด instruction overhead**: loop unrolling, หลีกเลี่ยง `%` และ `/`

6.3 Utilize bandwidth

- **Coalesced access**: thread ใน warp เข้าถึงตำแหน่งติดกัน — 32 threads × 4 B = 128 B = 4 sectors (32 B) ใช้ประโยชน์ 100%; stride 2 ใช้ได้ 50%; stride ≥ 8 ใช้ได้แค่ 12.5%
- **Data reuse ใน shared memory/register**: โหลดจาก global ครั้งเดียว ใช้หลายครั้ง
- **หลีกเลี่ยง bank conflict**: 32 banks กว้าง 4 B; thread ใน warp ที่เข้าถึง bank เดียวกัน (คนละ address) จะ serialize — ใช้ sequential addressing หรือ padding
- **ลด traffic ที่ไม่จำเป็น**: fuse kernel (เช่น map + reduce ใน kernel เดียว), ไม่เขียนผลกลางกลับ global ถ้าไม่ต้อง, ใช้ data type เล็กลง (`uchar` แทน `int`)
- **Vectorized load** (`float4`) ลดจำนวน memory instruction

6.4 Occupancy: วิธีคำนวณ

จำนวน block ต่อ SM = min ของข้อจำกัดต่อไปนี้:

$$\left\lfloor \frac{\text{max threads/SM}}{\text{threads/block}} \right\rfloor, \left\lfloor \frac{\text{registers/SM}}{\text{regs/thread} \times \text{threads/block}} \right\rfloor, \left\lfloor \frac{\text{shared mem/SM}}{\text{shared/block}} \right\rfloor, \text{ max blocks/SM}$$

occupancy = (blocks/SM × threads/block) / (max threads/SM). Occupancy สูงช่วย latency hiding แต่ไม่จำเป็นต้อง 100% — ให้ดูจาก bandwidth/throughput ที่วัดได้จริง

6.5 Checklist สำหรับตอบข้อสอบ "เสนอ idea fine-tune"

แนวข้อสอบ

ตอบเป็นโครงสร้าง **ปัญหา** → **principle** → **วิธีแก้** → **วัดผลอย่างไร** เช่น

1. *ปัญหา*: kernel reduce ได้ effective bandwidth 15% ของ peak
2. *Principle*: reduce เป็น memory-bound → ต้อง utilize bandwidth
3. *สาเหตุ/วิธีแก้*: global access ไม่ coalesced, thread ครึ่งหนึ่ง idle ตั้งแต่ step แรก → ใช้ grid-stride loop (coalesced + coarsening), first-add-during-load, warp shuffle
4. *วัดผล*: เวลา (cudaEvent), GB/s เทียบ peak, Nsight: `dram__throughput`, divergence

7 สรุปสูตรและตารางอ้างอิงเร็ว

Algorithm	Work	Step	Work-optimal?	Model
Sequential reduce/scan	$n-1$	$n-1$	—	RAM
Tree reduce	$O(n)$	$O(\log n)$	ใช่	EREW
Hillis–Steele scan	$O(n \log n)$	$\log n$	ไม่	EREW (double buffer)
Blelloch scan	$O(n)$ ($\approx 2n$)	$2 \log n$	ใช่	EREW
Recursive scan (pram.pdf)	$O(n)$	$O(\log n)$	ใช่	EREW
Histogram (matrix)	$O(nm)$	$O(\log n)$	ไม่	EREW
Histogram (cascaded)	$O(n)$	$O(m + \log n)$	ใช่	EREW
Pointer jumping (list)	$O(n \log n)$	$O(\log n)$	ไม่	CREW
Max (Common CRCW, n^2 proc)	$O(n^2)$	$O(1)$	ไม่	CRCW
Filter / pack / split	$O(n)$	$O(\log n)$	ใช่	EREW
Flatten (m = output)	$O(n+m)$	$O(\log n)$	ใช่	EREW
Binary search	$O(\log n)$	$O(\log n)$	ใช่	RAM
$\sqrt{n}$-way search	$O(\sqrt{n})$	$O(1)$	ไม่	CREW
Merge (rank ทุกตัว)	$O(m \log m)$	$O(\log m)$	ไม่	CREW
Merge (merge path)	$O(m)$	$O(\log m)$	ใช่	CREW
K-th smallest (random)	$O(n)$ exp.	$O(\log^2 n)$ whp	ใช่	EREW

สูตรที่ต้องจำ

$$S_p = \frac{T^*(n)}{T_p}, \quad C_p = pT_p, \quad E_p = \frac{S_p}{p}, \qquad \text{optimal speedup} \iff C_p = \Theta(T^*(n))$$

Brent: $\max\left(\frac{W}{p}, S\right) \le T_p \le \frac{W}{p} + S$

Work-optimal: $W = \Theta(T^*(n))$; optimal speedup ได้เมื่อ $p = O(T^*(n)/S)$

Sum บน PRAM: $T_p = O(n/p + \log p)$, optimal เมื่อ $p = O(n/\log n)$

$$BW_{\text{eff}} = \frac{B_r + B_w}{t}, \qquad AI = \frac{\text{FLOPs}}{\text{bytes}}$$